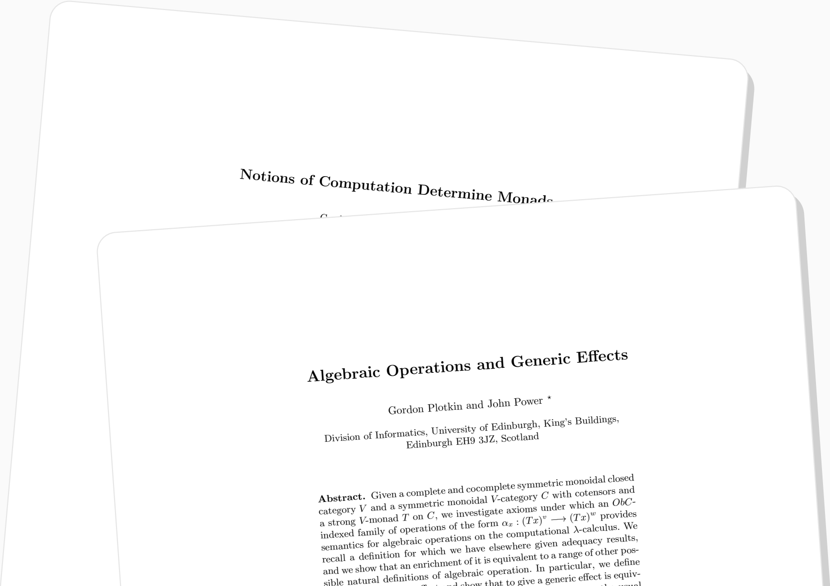
Sound Operational Game Semantics for generative algebraic effects and handlers

Hamza Jaafar, Guilhem Jaber

September 27, 2025

Nantes Université, LS2N, INRIA Gallinette

Algebraic effects — seminal papers



Notions of Computation Determine Monads

Algebraic Operations and Generic Effects

Gordon Plotkin and John Power *

Division of Informatics, University of Edinburgh, King's Buildings, Edinburgh EH9 3JZ, Scotland

Abstract. Given a complete and cocomplete symmetric monoidal closed category V and a symmetric monoidal V-category C with cotensors and a strong V-monad T on C, we investigate axioms under which an ObC-indexed family of operations of the form $\alpha_x : (Tx)^v \longrightarrow (Tx)^w$ provides semantics for algebraic operations on the computational λ-calculus. We recall a definition for which we have elsewhere given adequacy results, and we show that an enrichment of it is equivalent to a range of other possible natural definitions of algebraic operation. In particular, we define

Algebraic effects and handlers

Main idea: Computational effects are *specified* by an *algebraic theory*.

- *Signature*:

$$\textbf{choose} : \tau \times \tau \twoheadrightarrow \tau \qquad \textbf{fail} : \mathbb{1} \twoheadrightarrow$$

- *Equations*

$$\textbf{choose}(\mathtt{t}, \mathtt{t}) \equiv \mathtt{t} \tag{Idem}$$

$$\textbf{choose}(\mathtt{t}, \mathtt{u}) \equiv \textbf{choose}(\mathtt{u}, \mathtt{t}) \tag{Sym}$$

$$\textbf{choose}(\textbf{choose}(\mathtt{t}, \mathtt{u}), \mathtt{e}) \equiv \textbf{choose}(\mathtt{e}, \textbf{choose}(\mathtt{t}, \mathtt{u})) \tag{Asso}$$

Constructors of effects, concretely

Every operation symbol **op** comes with an arity

$$\textbf{op} \; : \quad \tau \twoheadrightarrow \sigma$$

defining an algebraic operation

$$\overline{\textbf{op}}(\mathrm{v}, \kappa) \qquad \mathrm{v} : \tau \;\text{ and }\; \kappa : \upsilon \to \varphi$$

or equivalently[1], a *generic operation*

$$\textbf{op}\,\mathrm{v}$$

[1] by algebraicity, $\mathtt{E}[\textbf{op}\,\mathrm{v}] = \overline{\textbf{op}}(\mathrm{v}, \lambda x.\mathtt{E}[x])$

Handler as destructors of effects

A generalization of exception handlers (constructs such as **try** $\cdots$ **catch** or **try** $\cdots$ **with**) that can capture the *delimited continuation*[2].

$$
\begin{array}{rcll}
\mathtt{h} & = & \{\mathbf{ret}\, x \mapsto \mathtt{u}\} & (\textit{return clause}) \\
 & & \{\cdots, \overline{\mathbf{op}_i}(x, k) \mapsto \mathtt{t}_i, \cdots\} & (\mathbf{op}\ \textit{clauses})
\end{array}
$$

[2]Benton and Kennedy, "Exceptional Syntax".

Handler as destructors of effects

A generalization of exception handlers (constructs such as **try** $\cdots$ **catch** or **try** $\cdots$ **with**) that can capture the *delimited continuation*[2].

$$
\begin{array}{rll}
\mathtt{h} = & \{\mathbf{ret}\, x \mapsto \mathtt{u}\} & (\textit{return clause}) \\
& \{\cdots, \overline{\mathbf{op}_i}(x, k) \mapsto \mathtt{t}_i, \cdots\} & (\mathbf{op}\ \textit{clauses})
\end{array}
$$

$$
\begin{array}{lll}
\mathbf{handle}\,(\mathbf{ret}\ \mathtt{v})\,\mathbf{with}\,\mathtt{h} & \to_v & \mathtt{u}\{\mathtt{v}/x\} \\
\mathbf{handle}\,\mathtt{E}[\mathbf{op}\ \mathtt{v}]\,\mathbf{with}\,\mathtt{h} & \to_v & \mathtt{t}_i\{\mathtt{v}/x\}\{(\lambda y.\mathbf{handle}\,\mathtt{E}[y]\,\mathbf{with}\,\mathtt{h})/k\}
\end{array}
$$

when $\mathtt{E}$ does not handle **op**

[2] Benton and Kennedy, "Exceptional Syntax".

Effect interfaces are given by signatures. *e.g.* **I/O**:

$$\{\textbf{write} : \tau \twoheadrightarrow \mathbb{1}, \textbf{read} : \mathbb{1} \twoheadrightarrow \tau\}$$

What if we want multiple states holding values of type τ, multiple open file descriptors, etc., without forfeiting modularity?

The Eff[3] approach

$\Longrightarrow$ First-class identifier ι for each *instance* of an effect $\mathbb{E}$.

- Performing an effect: $\iota\#\mathsf{op}_{\mathbb{E}}\ \mathtt{v}$
- Handling an effect: $\mathtt{h} = \{\iota\#\mathsf{op}_{\mathbb{E}}\ p\,\kappa \mapsto \mathtt{t}\}$

$\Longrightarrow$ Dynamic generation:

$$\textbf{let}\, x \Leftarrow \textbf{new}\, \mathbb{E}\ \textbf{in handle}\, \mathtt{t}\ \textbf{with}\, \mathtt{h} \quad \rightarrow_v \quad \underset{\text{fresh instance } \iota}{\textbf{handle}\, \mathtt{t}\{\iota/x\}\ \textbf{with}\, \mathtt{h}\{\iota/x\}}$$

[3] Bauer and Pretnar, "Programming with algebraic effects and handlers".

Programming Language

Fine-grained[4] call-by-value + alebraic effects & handlers

$$
\begin{array}{llcl}
\text{values} & \mathtt{v}, \mathtt{w} & := & x \mid \lambda x : \tau.\mathtt{t} \mid \iota \\[1em]
\text{terms} & \mathtt{t}, \mathtt{u} & := & \textbf{ret}\ \mathtt{v} \mid \mathtt{v}\ \mathtt{v} \mid \textbf{match}\ \mathtt{v}\ \textbf{with}\ (\mathtt{P}_i \to \mathtt{u}_i)_{i \in I} \\
 & & \mid & \textbf{let}\, x \Leftarrow \mathtt{t}\ \textbf{in}\ \mathtt{u} \\
 & & \mid & \textbf{new}\, \mathbb{E} \mid \mathtt{v}\textbf{\#op}\, \mathtt{w} \mid \textbf{handle}\, \mathtt{t}\, \textbf{with}\, \mathtt{h} \\[1em]
\text{handlers} & \mathtt{h} & := & \{\textbf{ret}\, x \mapsto \mathtt{t}\} \mid \{\mathtt{v}\textbf{\#op}\ x\, k \mapsto \mathtt{t}\} \uplus \mathtt{h}
\end{array}
$$

[4] Levy, *Call-By-Push-Value: A Functional/Imperative Synthesis*.

Characterizing Contextual Equivalence

Contextual Equivalence

We consider a standard notion of contextual equivalence $\simeq_{ctx}$ whereby only termination is *observed*.

Definition (Observation)

$$\begin{gathered}\texttt{t} \text{ is terminating} \\ \textit{if and only if} \\ \exists \texttt{v} \ \textit{s.t} \ \ \texttt{t} \Downarrow_{\mathsf{op}} \textbf{ret} \ \texttt{v}\end{gathered}$$

Definition (Contextual equivalence)

$$\texttt{t} \simeq_{ctx} \texttt{u} :\Longleftrightarrow \ \forall \texttt{C}.\texttt{C}[\texttt{t}] \Downarrow_{\mathsf{op}} \text{iff } \texttt{C}[\texttt{u}] \Downarrow_{\mathsf{op}}$$

Two equivalent encodings of state

Consider the standard encoding of state

$$\mathtt{h}_{\mathit{state}}(\iota) := \begin{array}{l} \{ \quad \mathbf{ret}\, x \mapsto \lambda s.\, \mathbf{ret}\, x; \\ \qquad \iota\#\mathbf{get}\ \langle\rangle\ k \mapsto \lambda s. k\ s\ s; \\ \qquad \iota\#\mathbf{set}\ x\ k \mapsto \lambda s. k\ \langle\rangle\ s \\ \} \end{array}$$

[5] Biernacki et al., "Handle with care: relational interpretation of algebraic effects and handlers".

Two equivalent encodings of state

Consider the standard encoding of state

$$\mathtt{h}_{state}(\iota) := \quad \left\{ \begin{array}{l} \textbf{ret}\, x \mapsto \lambda s.\, \textbf{ret}\, x; \\ \iota\#\textbf{get}\ \langle\rangle\ k \mapsto \lambda s. k\ s\ s; \\ \iota\#\textbf{set}\ x\ k \mapsto \lambda s. k\ \langle\rangle\ s \end{array} \right.$$
$$\}$$

Another elegant encoding due to Biernacki et al.[5]

$$\mathtt{h}_{init}(\iota) := \{\iota\#\textbf{get}\ \langle\rangle\ k \mapsto k\ 0\}$$

$$\mathtt{h}_{set}(\iota) := \{\iota\#\textbf{set}\ {\color{blue}x}\ k \mapsto \textbf{handle}\ k\ \langle\rangle\ \textbf{with}\ \{\iota\#\textbf{get}\ \langle\rangle\ y \mapsto y\ {\color{blue}x}\}\}$$

[5] Biernacki et al., "Handle with care: relational interpretation of algebraic effects and handlers".

Interplay of generativity and contextual equivalence

$$(\textbf{handle}\,\mathtt{t}\,\textbf{with}\,\mathtt{h}_{state}(\iota))\,0 \\ \simeq_{ctx} \\ \textbf{handle}\,\mathtt{t}\,\textbf{with}\,\mathtt{h}_{init}\cdot\mathtt{h}_{set}(\iota) \tag{1}$$

Interplay of generativity and contextual equivalence

$$(\textbf{handle}\ \mathtt{t}\ \textbf{with}\ \mathtt{h}_{state}(\iota))\ 0 \\ \simeq_{ctx} \\ \textbf{handle}\ \mathtt{t}\ \textbf{with}\ \mathtt{h}_{init} \cdot \mathtt{h}_{set}(\iota) \tag{1}$$

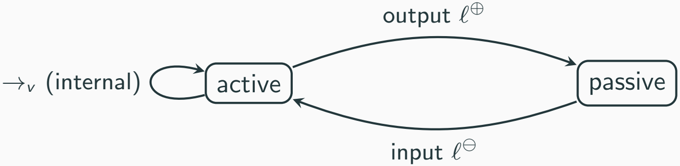
$$\textbf{let}\ x \Leftarrow \textbf{new}\ \mathbb{E}\ \textbf{in} \\ (\textbf{handle}\ \mathtt{t}\ \textbf{with}\ \mathtt{h}_{state}(x))\ 0 \\ \not\simeq_{ctx} \\ \textbf{handle}\ \mathtt{t}\ \textbf{with}\ \mathtt{h}_{init} \cdot \mathtt{h}_{set}(\iota) \tag{2}$$

Operational game semantics model

Operational Game Semantics *OGS*

Trace semantics following the operational evaluation of a program (Proponent) and tracing its interaction with its environment (Opponent).

$\Longrightarrow$ Interaction is modelled by a bi-partite transition system :



Operational Game Semantics (cont.)

Normal Forms:

$$\mathtt{n} ::= \mathtt{E}[x\ \mathtt{v}] \mid \textbf{ret}\ \mathtt{v}$$

- $\textbf{ret}\ \mathtt{v}$ calls for an answer.
- $\mathtt{E}[x\ \mathtt{v}]$ calls for a question.

$\Longrightarrow$ Labels:

$$\ell ::= \langle \textbf{ret}\ \mathtt{A} \mid c \rangle \mid \langle x\ \mathtt{A} \mid c \rangle$$

where c is an abstract continuation.

Example 1

Let's consider the trace of

$$f\ (\lambda x.5)$$

representing the interaction with the environment given by the context represented by c

$$\textbf{let}\ f \Leftarrow (\lambda g.g\ \mathtt{v}; \textbf{ret}\ \mathtt{tt})\ \textbf{in}\ []$$

$$\langle f\ g \mid c \rangle^{\oplus}$$

Example 1

Let's consider the trace of

$$f\ (\lambda x.5)$$

representing the interaction with the environment given by the context represented by c

$$\textbf{let}\ f \Leftarrow (\lambda g.g\ \mathtt{v};\textbf{ret}\ \ \mathtt{tt})\ \textbf{in}\ []$$

$$\langle f\ g \mid c\rangle^{\oplus}\ \langle g\ \mathtt{A} \mid d\rangle^{\ominus}$$

Example 1

Let's consider the trace of

$$f\ (\lambda x.5)$$

representing the interaction with the environment given by the context represented by c

$$\textbf{let}\ f \Leftarrow (\lambda g.g\ \mathtt{v}; \textbf{ret}\ \mathtt{tt})\ \textbf{in}\ []$$

$$\langle f\ g \mid c\rangle^{\oplus}\ \langle g\ \mathtt{A} \mid d\rangle^{\ominus}\ \langle \textbf{ret}\ 5 \mid d\rangle^{\oplus}$$

Example 1

Let's consider the trace of

$$f\ (\lambda x.5)$$

representing the interaction with the environment given by the context represented by c

$$\textbf{let}\ f \Leftarrow (\lambda g.g\ \mathtt{v}; \textbf{ret}\ \ \mathtt{tt})\ \textbf{in}\ []$$

$$\langle f\ g \mid c\rangle^{\oplus}\ \langle g\ \mathtt{A} \mid d\rangle^{\ominus}\ \langle \textbf{ret}\ 5 \mid d\rangle^{\oplus}\ \langle \textbf{ret}\ \mathtt{tt} \mid c\rangle^{\ominus}$$

The denotation $[\![\mathtt{t}]\!]_{\mathsf{ogs}}$ of a given program $\mathtt{t}$ is the set of all the traces it generates, *i.e.* all its execution runs against all possible environments.

Definition (trace equivalence)

$$\mathtt{t} \simeq_{tr} \mathtt{u} \quad :\Longleftrightarrow \quad [\![\mathtt{t}]\!]_{\mathsf{ogs}} = [\![\mathtt{u}]\!]_{\mathsf{ogs}}$$

OGS model for algebraic effects and handlers

Algebraic effects introduce new normal forms:

$$\mathtt{n} ::= \cdots \mid \mathtt{E}[\iota\#\mathbf{op}\,\mathtt{v}] \qquad \text{when } \iota\#\mathbf{op} \notin \mathbf{hdl}(\mathtt{E})$$

$\Longrightarrow$ It requires extending the interaction interface with new moves that account for observable effectful actions.

OGS model for algebraic effects and handlers

Algebraic effects introduce new normal forms:

$$\mathtt{n} ::= \cdots \mid \mathtt{E}[\iota\#\mathbf{op}\,\mathtt{v}] \qquad \text{when } \iota\#\mathbf{op} \notin \mathbf{hdl}(\mathtt{E})$$

$\Longrightarrow$ It requires extending the interaction interface with new moves that account for observable effectful actions.

But, what counts as *observable*?

Accommodating the *OGS* model for effect name disclosure

When the program performs an effect

$$\iota\texttt{\#op v}$$

$\Longrightarrow$ *What is the disclosure status of the instance ι?*

Accommodating the *OGS* model for effect name disclosure

When the program performs an effect

$$\iota\texttt{\#op v}$$

$\Longrightarrow$ *What is the disclosure status of the instance ι?*

- **Public**: Opponent could *potentially* handle the effect.
- **Private**: Opponent can only forward the effect to any enclosing Player's handling context.

Accommodating the *OGS* model for effect name disclosure

$$\mathtt{n} = \cdots \mid \mathtt{E}[\iota\#\mathbf{op}\ \mathtt{v}] \qquad \text{when } \ \iota\#\mathbf{op} \notin \mathbf{hdl}(\mathtt{E})$$

$\Longrightarrow$ New interactive moves capturing the delimited continuation $\mathtt{E}$ and the effect:

- **private effect** $\langle \kappa[e] \mid d \rangle$
- **public effect** $\langle \kappa[\iota\#\mathbf{op}\ \mathtt{A}] \mid c \rangle$

where κ is an abstract delimited continuation.

Example 2

Now we look at how the term

$$\textbf{handle}\, f\, (\lambda x.\, \iota\#\textbf{op}\, \langle\rangle)\, \textbf{with}\, \{\iota\#\textbf{op}\ x\, k \mapsto k\ 5\}$$

interacts with the same environement

$$\textbf{let}\, f \Leftarrow (\lambda g.g\ \ \mathrm{v}; \textbf{ret}\ \ \mathtt{tt})\ \textbf{in}\ []$$

$$\langle f\ g \mid c\rangle^{\oplus}$$

Example 2

Now we look at how the term

$$\textbf{handle}\ f\ (\lambda x.\ \iota\#\textbf{op}\ \langle\rangle)\ \textbf{with}\ \{\iota\#\textbf{op}\ x\ k \mapsto k\ 5\}$$

interacts with the same environement

$$\textbf{let}\ f \Leftarrow (\lambda g.g\ \mathtt{v}; \textbf{ret}\ \mathbb{tt})\ \textbf{in}\ []$$

$\langle f\ g \mid c\rangle^{\oplus}\ \langle g\ \mathtt{A} \mid d\rangle^{\ominus}$

Example 2

Now we look at how the term

$$\textbf{handle}\ f\ (\lambda x.\ \iota\#\textbf{op}\ \langle\rangle)\ \textbf{with}\ \{\iota\#\textbf{op}\ x\ k \mapsto k\ 5\}$$

interacts with the same environement

$$\textbf{let}\ f \Leftarrow (\lambda g.g\ \mathtt{v}; \textbf{ret}\ \ \mathbb{tt})\ \textbf{in}\ []$$

$$\langle f\ g \mid c\rangle^{\oplus}\ \langle g\ \mathtt{A} \mid d\rangle^{\ominus}\ \langle \kappa[e] \mid d\rangle^{\oplus}$$

Example 2

Now we look at how the term

$$\textbf{handle}\, f\ (\lambda x.\ \iota\#\textbf{op}\ \langle\rangle)\ \textbf{with}\ \{\iota\#\textbf{op}\ x\ k \mapsto k\ 5\}$$

interacts with the same environement

$$\textbf{let}\ f \Leftarrow (\lambda g.g\ \ \mathtt{v};\textbf{ret}\ \ \mathtt{tt})\ \textbf{in}\ []$$

$$\langle f\ g \mid c\rangle^{\oplus}\ \langle g\ \mathtt{A} \mid d\rangle^{\ominus}\ \langle \kappa[e] \mid d\rangle^{\oplus}\ \langle \kappa_d \cdot \kappa[e] \mid c\rangle^{\ominus}$$

Example 2

Now we look at how the term

$$\textbf{handle}\ f\ (\lambda x.\ \iota\#\textbf{op}\ \langle\rangle)\ \textbf{with}\ \{\iota\#\textbf{op}\ x\ k \mapsto k\ 5\}$$

interacts with the same environement

$$\textbf{let}\ f \Leftarrow (\lambda g.g\ \mathtt{v}; \textbf{ret}\ \ \mathtt{tt})\ \textbf{in}\ []$$

$$\langle f\ g \mid c\rangle^{\oplus}\ \langle g\ \mathtt{A} \mid d\rangle^{\ominus}\ \langle \kappa[e] \mid d\rangle^{\oplus}\ \langle \kappa_d \cdot \kappa[e] \mid c\rangle^{\ominus}\ \langle \kappa_d[\textbf{ret}\ 5] \mid c'\rangle^{\oplus}$$

Example 2

Now we look at how the term

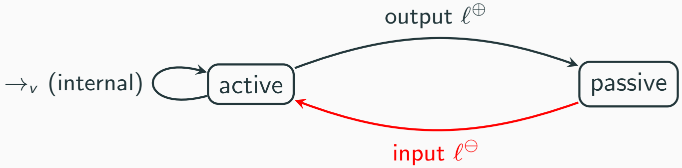
$$\textbf{handle}\ f\ (\lambda x.\ \iota\#\textbf{op}\ \langle\rangle)\ \textbf{with}\ \{\iota\#\textbf{op}\ x\ k \mapsto k\ 5\}$$

interacts with the same environement

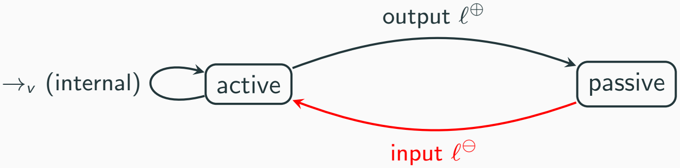
$$\textbf{let}\ f \Leftarrow (\lambda g.g\ \mathtt{v}; \textbf{ret}\ \ \mathtt{tt})\ \textbf{in}\ []$$

$$\langle f\ g \mid c\rangle^{\oplus}\ \langle g\ \mathtt{A} \mid d\rangle^{\ominus}\ \langle \kappa[e] \mid d\rangle^{\oplus}\ \langle \kappa_d \cdot \kappa[e] \mid c\rangle^{\ominus}\ \langle \kappa_d[\textbf{ret}\ 5] \mid c'\rangle^{\oplus}\ \langle \textbf{ret}\ \mathtt{tt} \mid c'\rangle^{\ominus}$$

Constraining Opponent's behaviour

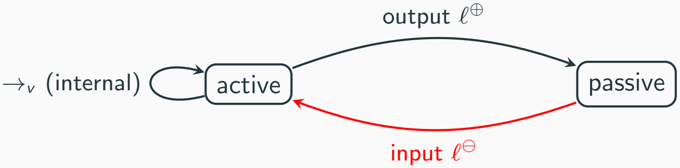


Constraining Opponent's behaviour



No constraints $\implies$ Opponent can simulate undelimited control (à la **call**/**cc**), and hence can discriminate more programs than any *real* context.

Constraining Opponent's behaviour



No constraints $\implies$ Opponent can simulate undelimited control (à la **call**/**cc**), and hence can discriminate more programs than any *real* context.

Well-bracketed $\implies$ Opponent cannot simulate effect handlers, hence its discriminating powers are no match to *real* program contexts.

Modelling delimited control flow

The capturing of a delimited continuation via a handler corresponds to the transition

$$\textbf{handle}\,\mathrm{E}_{n+1}[\kappa_n[\mathrm{E}_n[\cdots\kappa_0[\mathrm{E}_0[\iota\#\textbf{op}\,\mathtt{v}]]\cdots]]]\,\textbf{with}\,\mathtt{h} \rightarrow_v \cdots$$

where $\iota\#\textbf{op} \in \textbf{hdl}(\mathtt{h})$

...hence, the *actual* resumable continuation is

$$\boxed{\lambda x.(\textbf{handle}\,\mathrm{E}_{n+1}[\kappa_n[\mathrm{E}_n[\cdots\kappa_0[\mathrm{E}_0[\textbf{ret}\,x]]\,\textbf{with}\,\mathtt{h})}$$

Modelling delimited control flow

The capturing of a delimited continuation via a handler corresponds to the transition

$$\textbf{handle}\,\mathrm{E}_{n+1}[\kappa_n[\mathrm{E}_n[\cdots\kappa_0[\mathrm{E}_0[\iota\#\textbf{op}\,\mathtt{v}]]\cdots]]]\,\textbf{with}\,\mathtt{h} \to_v \cdots$$

where $\iota\#\textbf{op} \in \textbf{hdl}(\mathtt{h})$

…hence, the *actual* resumable continuation is

$$\boxed{\lambda x.(\textbf{handle}\,\mathrm{E}_{n+1}[\kappa_n[\mathrm{E}_n[\cdots\kappa_0[\mathrm{E}_0[\textbf{ret}\,x]]\,\textbf{with}\,\mathtt{h})}$$

$\Longrightarrow$ Stack discipline can be disrupted, but the fragments κ_i of the captured continuation can only be used in the order in which they have been disclosed.

Soundness of the OGS model

Definition 4 (Trace Equivalence)

$$\mathtt{t} \simeq_{tr} \mathtt{u} \;\; :\Longleftrightarrow \quad [\![\mathtt{t}]\!]_{\mathsf{ogs}} = [\![\mathtt{u}]\!]_{\mathsf{ogs}}$$

Theorem 5 (Soundness)

$$\simeq_{tr} \quad \subseteq \quad \simeq_{ctx}$$

Conclusion

- Contextual equivalence is more subtle in the presence of generativity of first-class effect instances.
- Extending the OGS model to account for effectful behaviour.
- Relaxing trace equivalence to coincide with the contextual one.

QUESTIONS?

Extra slides

Algebraic effects and handlers

- Impure behaviour given by operations on computations[6] (e.g chose for non-deterministic choice, raise for exceptions...)
- Impure behaviour is described by an equational theory on these operations
- Account for monadic effects whose behaviour is independent of the current evaluation context

$$\mathrm{chose}(\mathtt{E}[\mathtt{t}], \mathtt{E}[\mathtt{u}]) \sim_{\mathsf{op}} \mathtt{E}[\mathrm{chose}(\mathtt{t}, \mathtt{u})]$$

- Easier to structure compared to combining monadic effects.
- Handlers arise as homomorphisms between models of such algebraic theories.

[6] Gordon Plotkin and John Power. **"Semantics for algebraic operations".** In: *Electronic Notes in Theoretical Computer Science* 45 (2001), pp. 332–345.

Contextual Equivalence

$\mathtt{t}$ is terminating

if and only if

$\exists \mathtt{v}$ *s.t* $\mathtt{t} \Downarrow_{\mathsf{op}} \textbf{ret } \mathtt{v}$

Definition 6 (Contextual equivalence)

$$\mathtt{t} \simeq_{ctx} \mathtt{u} :\Longleftrightarrow \forall \mathtt{C}.\mathtt{C}[\mathtt{t}] \Downarrow_{\mathsf{op}} \text{iff } \mathtt{C}[\mathtt{u}] \Downarrow_{\mathsf{op}}$$

Disclosure and contextual equivalence

Consider the following variation of an example from[7]

$$f\ (\lambda x.5)$$

$$\simeq_{ctx}$$

$$\begin{array}{l}\textbf{let}\ y \Leftarrow \textbf{new}\ \mathbb{E}\ \textbf{in} \\ \textbf{handle} \\ \quad f\ (\lambda x.\ y\#\textbf{op}\ \langle\rangle) \\ \textbf{with}\ \{y\#\textbf{op}\ x\ \kappa \mapsto \kappa\ 5\}\end{array}$$

[7] Dariusz Biernacki et al. **"Handle with care: relational interpretation of algebraic effects and handlers".** In: *Proceedings of the ACM on Programming Languages* 2.POPL (2017), pp. 1–30.

We need a coarser notion of trace equivalence in which

$$\langle f\ g \mid c\rangle^{\oplus}\ \langle g\ \mathtt{A} \mid d\rangle^{\ominus}\ \langle \mathbf{ret}\, d \mid 5\rangle^{\oplus}\ \langle \mathbf{ret}\, c \mid \mathtt{tt}\rangle^{\ominus}$$
$$\simeq_{tr}$$
$$\langle f\ g \mid c\rangle^{\oplus}\ \langle g\ \mathtt{A} \mid d\rangle^{\ominus}\ \langle \kappa[e] \mid d\rangle^{\oplus}\ \langle \varkappa \cdot \kappa[e] \mid c\rangle^{\oplus} \langle k_d\, 5 \mid c'\rangle^{\oplus}\ \langle \mathbf{ret}\, c' \mid \mathtt{tt}\rangle^{\ominus}$$

Disclosure and contextual equivalence (cont.)

Now consider a variation of the previous example:

$$\textbf{let}\ y \Leftarrow \textbf{new}\ \mathbb{E}\ \textbf{in}\ g\ y; f\ (\lambda x.5)$$

$$\not\simeq_{ctx}$$

$$\begin{array}{l} \textbf{let}\ \ y\ =\!\textbf{new}\ \ \mathbb{E}\ \ \textbf{in} \\ \textbf{handle} \\ \quad g\ \ y\ ;\ f\ (\lambda x.\ y\#\textbf{op}\ \langle\rangle) \\ \textbf{with}\ \{y\#\textbf{op}\ x\ \kappa \mapsto \kappa\ 5\} \end{array}$$